程序设计原理

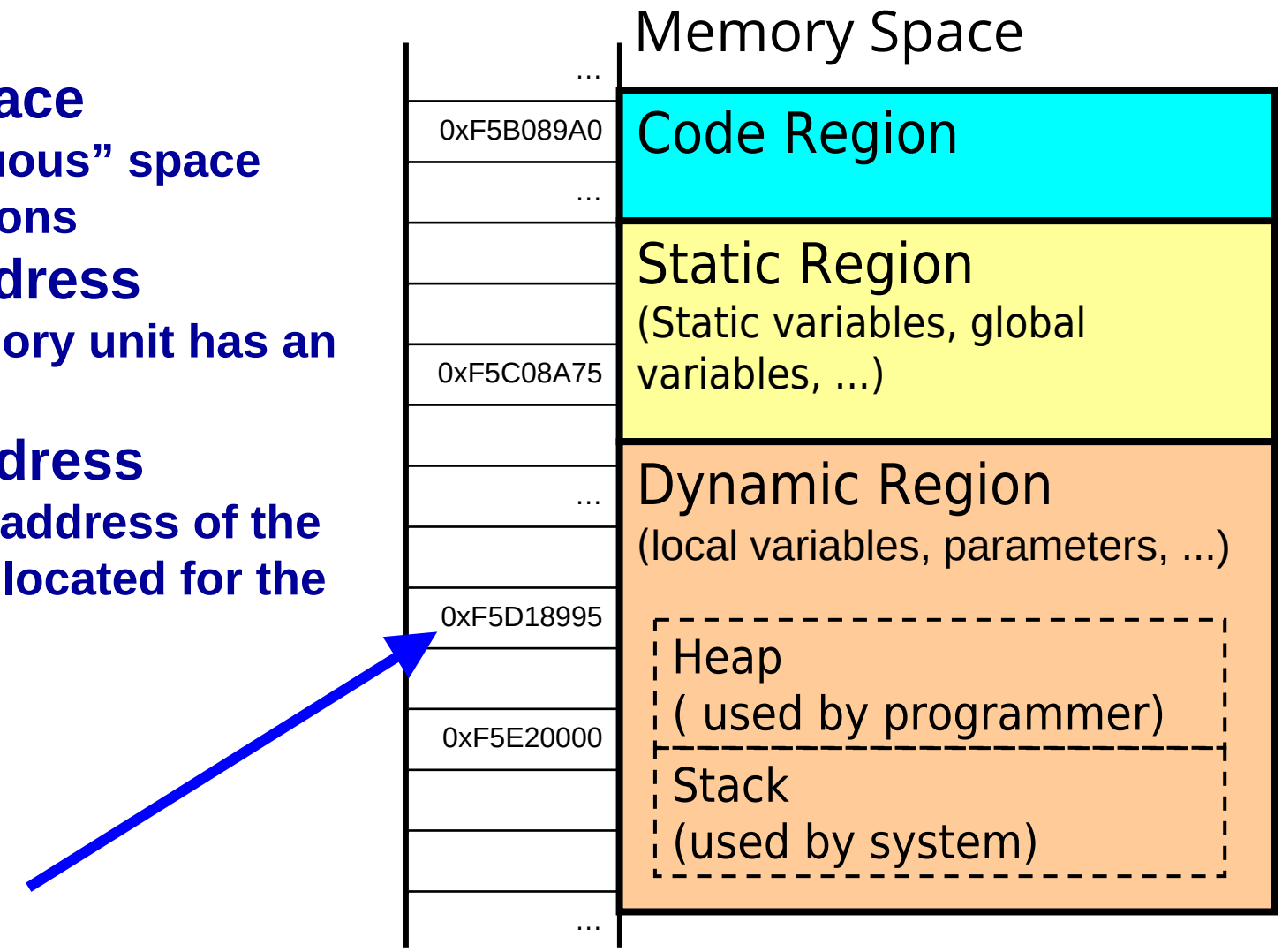
指针

任课教师： 喻　梅
于瑞国
王建荣
李雪威
赵满坤

Pointer Basics

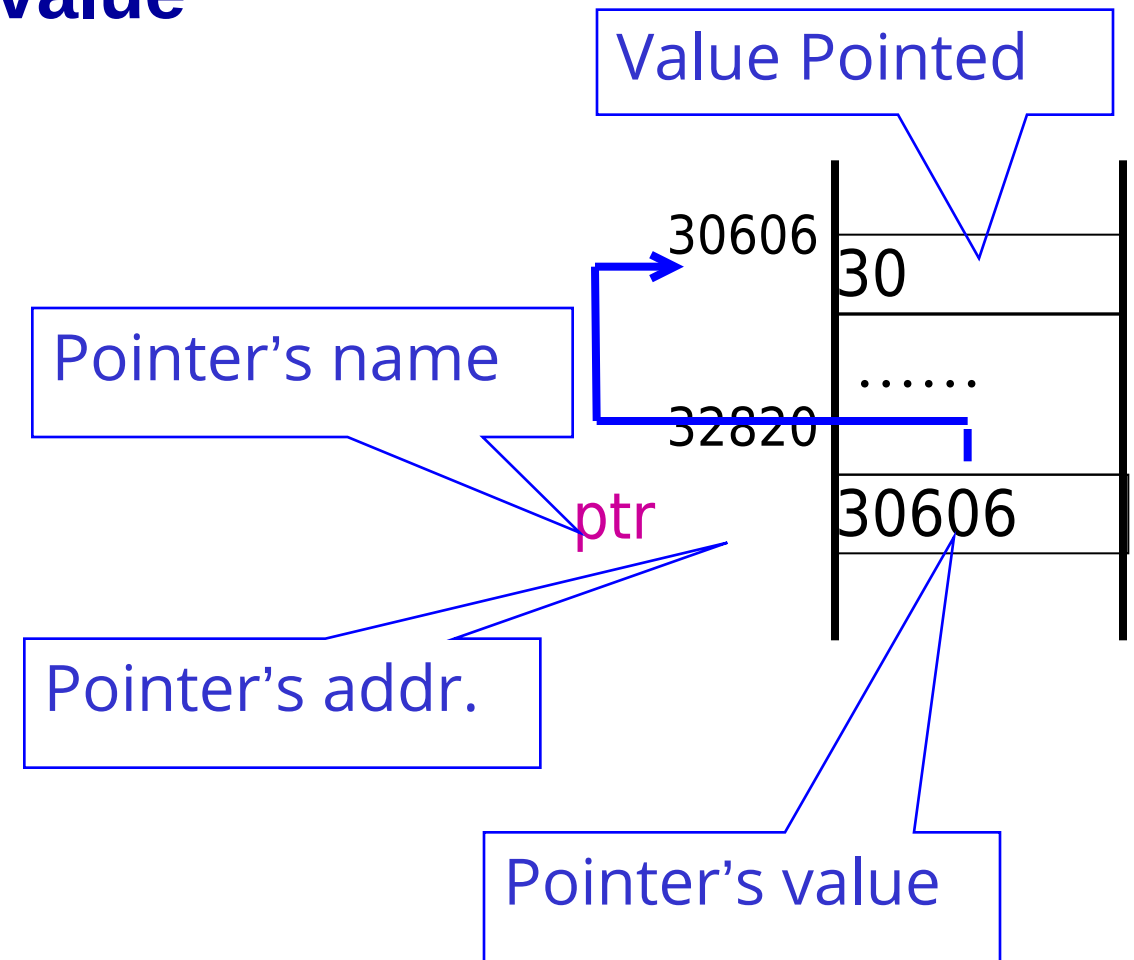
- **Memory space**
  - **A “continuous” space**
  - **Three regions**
- **Memory address**
  - **Each memory unit has an address**
- **Variable address**
  - **The initial address of the memory allocated for the variable**

int i=0;



What's a Pointer?

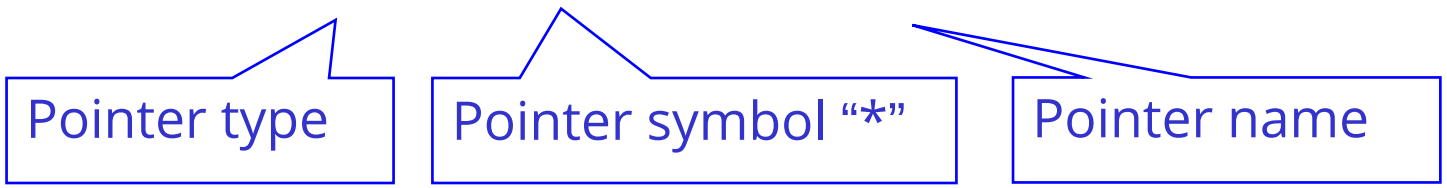
- **Pointer:**
  - **the variable to hold memory addresses as its value**



Declare a Pointer

- **Pointer is a composite data type**
- **Syntax:**

```
dataType * pointerName;
```



- **For example, a pointer named pCount that can point to an int value:**

```
int *pCount;
```

Initialize a Pointer

- **Implicit initialization**
  - **A local pointer is assigned an arbitrary value**
  - **A global pointer is assigned to NULL (pointing nothing)**
    - **NULL is in fact “00000000”**

Not recommended!
May cause fatal runtime error or accidental data modification!

Initialize a Pointer

- **Explicit initialization (recommended)**

```
int count=5;
int *pCount = &count;
int *ptr = NULL;
```

Address-of operator “&”. The value of “&i” is the address of i.

Indirect Reference

- ***Indirection:*** **referencing a value through a pointer.**

For example:

```
int count=5;
 int *pCount = &count;
count++; //direct reference
(*pCount)++; //indirect reference
```

Indirection operator “*”.
The value of “*ptr” is the valued pointed by ptr.

Indirect Reference

```
int age;
int *age_ptr = &age;
```

Then,

age_ptr 🔔 &age

*age_ptr 🔔 age

*age_ptr = 50 🔔 age = 50;

(*age_ptr)++; 🔔age++;

Null pointer

- A null pointer is a regular pointer of any pointer type which has a special value that indicates that it is not pointing to any valid reference or memory address. This value is the result of type-casting the integer value zero to any pointer type.

```
int * p;
p = 0; // p has a null pointer value
   p = NULL; or p = nullptr;//C++ 11
```

- A null pointer is a value that any pointer may take to represent that it is pointing to "nowhere", while a void pointer is a special type of pointer that can point to somewhere without a specific type. One refers to the value stored in the pointer itself and the other to the type of data it points to.

Operations of Pointer

- Operations on the address, not the value pointed
- Several specific operations
  - Assignment
  - Movement
  - Subtraction
  - Comparison
- The pointer type determines the unit of the operation results

For example:

p±n;// move n*sizeof(pionter type)

Comparing Pointers

- Relational operators (<, >=, etc.) can be used to compare addresses in pointers
- Comparing addresses in pointers is not the same as comparing contents pointed at by pointers:

```
if (ptr1 == ptr2)   // compares
                                    // addresses
if (*ptr1 == *ptr2) // compares
                                    // contents
```

Examples

```
int main()
{
    int a =10, b = 20;
    int *pa, *pb;
    pa = &a; //Assign the address directly
    pb = pa+2;

    cout <<"a in: "<< &a <<endl;
    cout <<"b in: "<< &b <<endl;

    cout <<"pa is: "<< pa <<endl;
    cout <<"pa points to: "<< *pa <<endl;
    cout <<"pb is: "<< pb <<endl;
    cout <<"pb points to: "<< *pb <<endl;
    if(pa != pb)
        cout <<"pa and pb are not equal!"<<endl;
    cout<<"pb - pa is "<<pb-pa<<endl;
}
```

```
a in: 0x29fee4
b in: 0x29fee0

pa is: 0x29fee4
pa points to: 10
pb is: 0x29feec
pb points to: 2752228
pa and pb are not equal!
pb - pa is 2
请按任意键继续. . .
```

Cautions

- The type of the address assigned to a pointer must be consistent with the pointer type.

  For example,

  ```
  int area = 1;
  double *pArea = &area;
  ```

- The white space besides "*" is optional

  ```
  int*ptr; int *ptr; int* ptr; int * ptr;
  ```

- One "*" for one pointer

  ```
  int *ptr1, *ptr2; //two pointers
  int* ptr1, ptr2; // one pointer, one integer
  ```

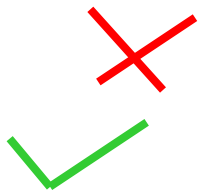
Some Special Pointers

- **Constant Pointer (指针常量)**
  - **A pointer whose value can not be changed**

```
double radius = 5;
double * const pValue = &radius;
(*pValue) = 3.0;
 pValue ++;
```

- **Pointer of Constant（常量指针）**
  - **A pointer that points to a constant**

```
double radius = 5;
const double * pValue = &radius;
 (*pValue) = 3.0;
  raidus = 3.0
```

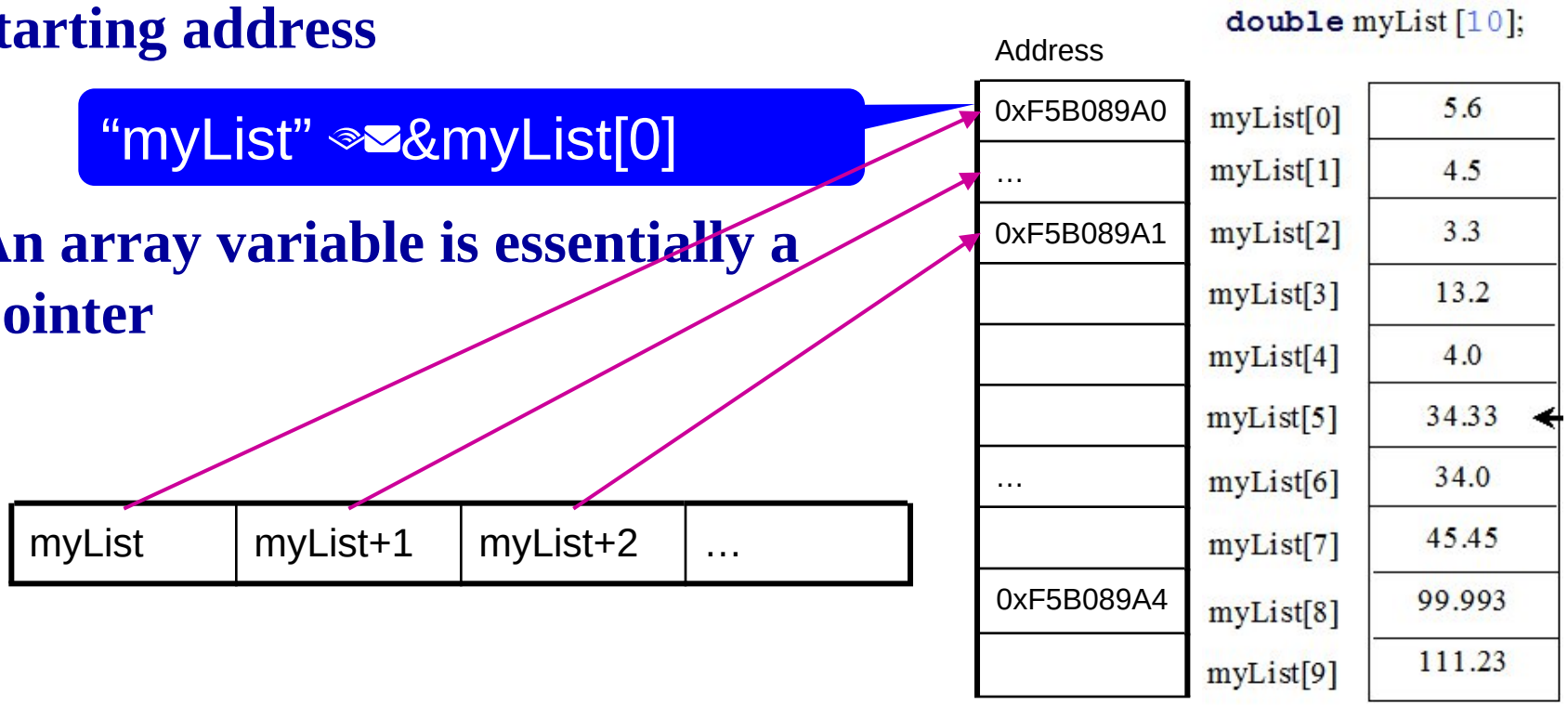


Pointer and Array

- **The name of an array represents the starting address**

“myList” ✉&myList[0]

- **An array variable is essentially a pointer**



Pointer and Array

- **Array variable is different from a pointer**
  - **Array name is a constant (not variable) address**

```
int array[10];
int*ptr=array;
for (k=0; k<5; k++){
  cout<< *(ptr++);
  cout<< *(array++);
}
```

Pointer and Array

```
#include <iostream>
int main( ){
   int a[5],i,sum1(0),sum2(0);
   cout<<"输入5个数:";
   for(i=0; i<5; i++)
       cin>>*(a+i);      //与cin>>a[i]相同。
   for(i=0,;i<5;i++)
       sum1+=*(a+i); //用数组名访问数组元素
   for(i=0,;i<5;i++)
       sum2+=a[i]; //用下标访问数组元素
   cout<<"sum1="<<sum1<<endl;
   cout<<"sum2="<<sum2<<endl;
   return 0;
 }
```

运行程序：

输入5个数:3 6 20 45 2
sum1=76
sum2=76

Pointer and Array

```
#include <iostream>
using namespace std;
int main(){
  int a[5]={10,20,30,40,50};
  int *p=a;       //等价于*p=&a[0];
  for(int i=0; i<5; i++)
     cout<<a[i]<<"  ";
  cout<<endl;
  for(i=0; i<5; i++)
     cout<<*(a+i)<<"  ";
  cout<<endl;
  for(i=0; i<5; i++)
     cout<<*(p+i)<<"  ";
  cout<<endl;
  for(i=0; i<5; i++)
     cout<<p[i]<<"  ";
  cout<<endl;
  p=a+3;
  cout<<*p<<endl;
  cout<<p-a<<endl;
}
```

程序执行结果：

```
10  20  30  40  50
10  20  30  40  50
10  20  30  40  50
10  20  30  40  50
40
3
```

该例进一步说明了数组与指针的关系。

执行结果中最后的 3 为 p-a的值，两地址相减结果为 p 指针与 a 数组首地址之间元素的个数。 而不一定是实际地址的数值差。

Pointer and Array

```
#include <iostream>

using namespace std;

int main(){
    int x[4] = {10,20,30,40};
    int *p = x;
    cout<<*++p<<endl;
    cout<<*p++<<endl;
    cout<<++*p<<endl;
    cout<<(*p)++<<endl;
    cout<<*p<<endl;
    return 0;
}
```

```
20
20
31
31
32
```

```
1  UPInt& UPInt::operator++()
2  {
3      *this += 1; // 增加
4      return *this; // 取回值
5  }
6  // postfix form: fetch and increment
7  const UPInt UPInt::operator++(int)
8  {
9      UPInt oldValue = *this; // 取回值
10     ++(*this); // 增加
11     return oldValue;
12 }
```

多维数组与指针

```
int a[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

A **two-dimensional array** is essentially a *one-dimensional array whose elements are themselves arrays.*

In memory, it is stored **row by row (row-major order)**.

a[0][0]	a[0][1]	a[0][2]	a[1][0]	a[1][1]	a[1][2]	a[2][0]	a[2][1]	a[2][2]	
↓	↓	↓	↓	↓	↓	↓	↓	↓	
1000	1004	1008	1012	1016	1020	1024	1028	1032	(addresses)

```
cout << a[i][j];          *(*(a + i) + j);   // same as a[i][j]
```

- a → the address of the first row (int (*)[3]) a → &a[0]
- a[i] → pointer to the i-th row (int*) a[i] → &a[i][0]
- a[i][j] → element at row *i*, column *j* a[i][j] → *(*(a + i) + j)

例： 使用指针法访问二维数组

```
#include <iostream>
using namespace std;

int main()
{
    int  a[3][4]={{1, 2, 3, 4,},
                  {5, 6, 7, 8},
                  {9, 10, 11, 12}};
    cout<<*(*a+0) << *a[0]<<a[0][0]<<endl;
    cout<<*(*(a+1)+0)<<*(a[1]+0)<<a[1][0]<<endl;
    cout<<*(*(a+2)+1)<<*(a[2]+1)<<a[2][1]<<endl;

    return 0;
}
```

程序执行结果：

1 1 1

5 5 5

10 10 10

在程序设计中既可以用下标法引用数组元素，也可以用指针（地址）法引用数组元素。

例： 使用指针法访问二维数组

```
#include <iostream>
using namespace std;
int main()
{
    int  a[3][4]={{1, 2, 3, 4,},
                  {5, 6, 7, 8},
                  {9, 10, 11, 12}};
    int (*p)[4] = a;//数组指针，与之对应，还有指针数组
    for(int i=0;i<3;i++){
        for(int j=0;j<3;j++){
            cout<<*(*(p+i)+j)<<" ";
        }
        cout << endl;
    }
    for(int i=0;i<3;i++){
        for(int j=0;j<3;j++){
            cout<<p[i][j]<<" ";
        }
        cout << endl;
    }

    return 0;
}
```